Lecture 2: Stacks, Queues, and Deques

CSI2110 – Data Structures and Algorithms

Lecture study notes

Source document: `../lec02-stacks,queues,deques.pdf`. These notes synthesize the source and add worked explanations. Page references use PDF page numbers, which can differ from printed slide numbers. Pseudocode is normalized for readability and boundary cases.

Contents

1 Abstract data types **2**

2 Stacks: last in, first out **2**

2.1 Array implementation 2

2.2 Singly linked implementation 2

3 Stack applications **3**

3.1 Reversing an array 3

3.2 Matching parentheses 3

3.3 Arithmetic expression evaluation 3

4 Queues: first in, first out **3**

4.1 Circular array implementation 3

4.2 Singly linked queue 4

5 Recent calls in a time window **4**

6 Deques: updates at both ends **4**

6.1 Circular-array formulas 5

6.2 Doubly linked implementation 5

7 Palindrome checking and operation tracing **5**

7.1 Solution to the lecture's stack/queue exercise 5

1 Abstract data types

Source: PDF pages 4–5. An **abstract data type (ADT)** specifies the stored objects, supported operations, and their effects. It does not specify the storage representation. A Java interface can express an ADT; a class implementing it supplies a concrete data structure.

The lecture's stack, queue, and deque ADTs return `null` for empty removal or inspection. This is a course convention; other APIs may throw exceptions. If null elements are allowed, a null return alone cannot distinguish an empty container from a stored null.

2 Stacks: last in, first out

Source: PDF pages 7–19. A stack inserts and removes at the same end, called the top. `push(e)` inserts; `pop()` removes and returns the top; `top()` inspects it. `size()` and `isEmpty()` give container information. Undo is a natural example: typing C, A, T and undoing twice leaves C.

2.1 Array implementation

Store entries in $S[0..t]$ of a capacity-N array. Initially $t = -1$, and the size is $t + 1$.

```
push(e):
    if t + 1 == N: report full-capacity error
    t = t + 1
    S[t] = e
pop():
    if t == -1: return null
    result = S[t]
    S[t] = null
    t = t - 1
    return result
```

Clearing the vacated reference permits garbage collection. Fixed-capacity operations all take $O(1)$; storage is $O(N)$. A growing array can occasionally incur $O(n)$ resizing, although doubling gives amortized $O(1)$ push.

2.2 Singly linked implementation

Make the head the top; maintain a size field.

```
push(e):
    head = new Node(e, head)
    size = size + 1
pop():
    if head == null: return null
    result = head.element
    head = head.next
    size = size - 1
    return result
```

All stack operations take $O(1)$; storage is $O(n)$. Using the tail as the top of a singly linked list would make pop slow.

3 Stack applications

Source: PDF pages 20–25.

3.1 Reversing an array

Push every entry in forward order, then pop entries back into the array in forward index order. LIFO reverses the order: $[4, 1, 2, 3] \mapsto [3, 2, 1, 4]$. Time is $\Theta(n)$; auxiliary stack space is $\Theta(n)$.

3.2 Matching parentheses

```
for each character c:
    if c is an opening delimiter:
        push(c)
    else if c is a closing delimiter:
        if stack is empty: return false
        if pop() is not c's matching opener: return false
return stack is empty
```

The stack contains unmatched opening delimiters in their encounter order. The next closer must match the most recent unmatched opener. Non-delimiter characters are ignored. Examples: `([]{})` is valid; `([)]`, `(()`, and `)(` are invalid. Time is $\Theta(n)$, with $O(n)$ auxiliary space in the worst case.

3.3 Arithmetic expression evaluation

Maintain a value stack V and an operator stack O. For the lecture's left-associative operators, before pushing a new operator, apply any waiting operator of higher or equal precedence. At the end, apply the remaining operators.

When applying an operator, pop the *right* operand first and the left operand second. Apply the operator to the left operand and then the right operand. Reversing these changes subtraction or division.

For $14 - 3 \times 2 + 7$, before pushing $+$, apply multiplication to get 6, then subtraction to get 8; the final result is 15. With each token pushed/popped a constant number of times, time is $O(n)$ and space $O(n)$. Parentheses need additional rules; the slides demonstrate precedence without them.

4 Queues: first in, first out

Source: PDF pages 27–39. A queue inserts at the rear and removes from the front. `enqueue(e)` inserts, `dequeue()` removes and returns the front, and `first()` inspects it. A shared printer serving jobs by arrival time uses FIFO.

4.1 Circular array implementation

Use capacity $N > 0$, front index f, and current size s. The logical entry at rank i is at $(f + i) \bmod N$.

$$\text{next insertion index} = (f + s) \bmod N, \qquad \text{last stored index} = (f + s - 1) \bmod N \quad (s > 0).$$

```
enqueue(e):
    if s == N: report full-capacity error
    Q[(f + s) mod N] = e
    s = s + 1
dequeue():
    if s == 0: return null
    result = Q[f]
    Q[f] = null
    f = (f + 1) mod N
    s = s - 1
    return result
```

Key idea. Front equal to next-insertion index can mean either empty or full. The size field distinguishes $s = 0$ from $s = N$.

Slide example: $N = 11, f = 7, s = 3$ holds A, B, C at indices $7, 8, 9$. Enqueue D at 10, E at 0, then dequeue A. Now $f = 8, s = 4$, and B, C, D, E occupy $8, 9, 10, 0$.

All fixed-capacity operations take $O(1)$ and allocated storage is $O(N)$. Circular indexing avoids shifting on dequeue.

4.2 Singly linked queue

The head is the front and the tail is the rear. Enqueue links a new node after the tail; dequeue removes the head. Maintain both references and a size field. For the first insertion set both ends; when removing the final element clear the tail. All operations take $O(1)$; storage is $O(n)$. Reversing front and rear would require tail deletion and incur $O(n)$ dequeue.

5 Recent calls in a time window

Source: PDF pages 40–41. Assume calls arrive with increasing timestamps. Keep timestamps in a queue, oldest first. The interval $[t - 3000, t]$ includes both endpoints.

```
ping(t):
    Q.enqueue(t)
    while not Q.isEmpty() and Q.first() < t - 3000:
        Q.dequeue()
    return Q.size()
```

For times $10, 500, 3010, 3501$, the results are $1, 2, 3, 2$. At 3010, timestamp 10 is retained because it equals the cutoff. At 3501, both 10 and 500 expire.

Additional analysis: one call may remove many entries and take $O(q)$ time if q timestamps are stored. Across m calls, each timestamp is enqueued once and dequeued at most once, so total queue work is $O(m)$ and amortized time is $O(1)$ per call. Space depends on calls inside the current window.

6 Deques: updates at both ends

Source: PDF pages 43–50. A deque (double-ended queue) supports these operations:

- Insert at either end: `addFirst(e)` and `addLast(e)`.
- Remove at either end: `removeFirst()` and `removeLast()`.
- Inspect either end: `first()` and `last()`.

- Report container information: `size()` and `isEmpty()`.

To implement a stack using a deque: push is `addLast`, pop is `removeLast`, and top is `last`. To implement a queue: enqueue is `addLast`, dequeue is `removeFirst`, and front inspection is `first`.

6.1 Circular-array formulas

Using front f and size s, test full/empty before modifying anything:

Operation	Update
`addFirst(e)`	$f \leftarrow (f-1+N) \bmod N$; write $D[f] \leftarrow e$; increment s.
`addLast(e)`	Write $D[(f+s) \bmod N] \leftarrow e$; increment s.
`removeFirst()`	Save and clear $D[f]$; advance $f \leftarrow (f+1) \bmod N$; decrement s.
`removeLast()`	Save and clear $D[(f+s-1) \bmod N]$; decrement s; leave f unchanged.

Adding N before modulo avoids a negative index in Java's remainder arithmetic. All fixed-capacity operations are $O(1)$; space is $O(N)$.

6.2 Doubly linked implementation

Use header and trailer sentinels. Insert/remove immediately after the header or immediately before the trailer. Direct previous and next links make all end operations $O(1)$. Space is $O(n)$. A singly linked list would not provide constant-time removal at both ends.

7 Palindrome checking and operation tracing

Source: PDF pages 51–52. To check a palindrome, put characters into a deque and repeatedly compare `removeFirst()` with `removeLast()` while at least two remain. Stop on a mismatch; zero or one remaining character is valid. This takes $\Theta(n)$ time and $\Theta(n)$ auxiliary space. RADAR is a palindrome. Normalizing spaces or case is an optional preprocessing choice, not specified by the lecture.

7.1 Solution to the lecture's stack/queue exercise

Stack entries below are listed *bottom to top*; queue entries are *front to rear*.

Checkpoint	Stack	Queue
After pushing/enqueuing 0 through 4	$[0,1,2,3,4]$	$[0,1,2,3,4]$
After four pops enqueued	$[0]$	$[0,1,2,3,4,4,3,2,1]$
After two dequeues pushed	$[0,0,1]$	$[2,3,4,4,3,2,1]$

Study checklist Be able to explain LIFO versus FIFO, trace circular wraparound, distinguish full from empty, choose suitable linked-list endpoints, and justify time and space for the application algorithms.